

Python Notes

Basics

Keywords are words which has special meaning and predefined functionality. Thus, these words cannot be used for any other purpose, such as variable or function names. In the code below, some important keywords (and, def, if, elif, else, for) are demonstrated. The full list of keywords is also provided.

```
import keyword
def my_function():
    for x in range(1, 6):
        if x > 1 and x < 5:
            print("middle")
        elif x == 1:
            print("beginning")
        else:
            print("end")
my_function()
print(keyword.kwlist)
```

```
beginning
middle
middle
middle
end
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else
```

An **identifier** is a user-created name given to a variable, function, class, or module in order to identify and use it, as shown below. The function is given the identifier 'sum' and we define its parameter variables as 'x' and 'y'. By giving these objects names, we can use them when calling the function and printing the sum value.

```
def sum(x, y):
    print(x + y)
sum(10, 7)
```

```
17
```

In Python, **comments** are written using hashtags or three single/double quotes. Comments are often used to make the code more readable and explain difficult sections. Hashtags are used for short, single line notes, while three quotes are for multi-line explanations. Since the function name 'new_function' below does not explain its purpose, a comment can do so instead. In the rest of the code, single line comments explain the purpose of a certain line

```
'''The function below finds the square of the sum of two numbers x and y.
In other words, it prints (x+y)^2'''
def new_function(x, y):
    sum = x + y
    print(sum*sum)
new_function(-1, 1) # this function call will print the result 0
# new function(2, 5) this function call will not print anything
```

```
0
```

Statements are basic instructions which the interpreter can execute that we write to complete a task. Examples of statements include imports, printing, assignment, conditionals, loops. Statements are usually a single line, but can be extended using the character '\'.

```
# an import statement
import keyword
# assignment stament continued to two lines
x = 1 + 2 + \
2 + 4 + 5 + 6
# conditional statement
if(x == 20):
    print("yay!") # print statement
```

```
# looping statement for x in range(1, 2):
    print("no!") # print statement
```

```
→ yay!
no!
```

In Python, **indentation** is used to define blocks of code. This is how the interpreter knows that code belongs in the body of an if-statement, within a function definition, or in a loop. Failing to indent will cause an indentation error.

```
x = 4
while x > 1:
    x = x - 1
    print(x) # since the loop was indented correctly, "3 2 1" will be printed
```

```
y = 4
while y > 1:
    y = y - 1
print(y) # since the loop is not indented, the same code will print just "1"
```

```
→ 3
   2
   1
   1
```

A **docstring** is the first line(s) of a function, class, or module and documents the usage and purpose in three quotes. Unlike a regular comment, docstrings are stored as an attribute of their associated object and can be accessed.

```
def mult(x, y):
    """
    Multiplies two numbers and returns the product.

    Args:
        x: first number
        y: second number

    Returns:
        The product the the two parameters
    """
    return x*y
prod = mult(5, 2)
print(prod)
print(mult.__doc__) # we can access the docstring with this call
```

```
→ 10
```

```
    Multiplies two numbers and returns the product.
```

```
    Args:
        x: first number
        y: second number
```

```
    Returns:
        The product the the two parameters
```

Strings in Python are indicated through single or double quotes. For multi-line strings, three quotes can be used.

We can find the length of a string using the `len()` function and search for certain characters within it using the keyword `'in'`.

Strings have both **forward** and **backward indexing**. In forward indexing the first character is in position 0 and can be accessed with `str[0]`, the second character is in position 1, and so on. For backward indexing, the last character is in position -1, the second to last is in -2, and this continues in decreasing values.

Using **string slicing**, we can get a range of characters within a string. For the first 5 characters, we can use `str[:5]`. `str[5:]`, however, will get the characters from position 5 to the end of the string. To retrieve the characters from position 1 to position 5, we use `str[1,6]`, since the end of the range is not inclusive. Splicing can also be done with backward indexing.

To modify a string, the `replace(str1, str2)` method can be used to replace all occurrences of `str1` in the string with `str2`. The string can also be divided around a certain phrase using the `split(str)` method. Furthermore, the `upper()` and `lower()` functions are used to convert the entire string to uppercase or lowercase, respectively. The `strip()` function is used to remove all whitespace from the beginning and end of a string. To conjoin two strings together, we simply use the addition operator `'+'`.

```
str = " Hello, World "  
print(str)  
str1 = "Say" + str[:7]  
# this will print "Say Hello," the first five characters with "Say" concatenated  
print(str1)  
str2 = str[-7:-1]  
# since we used backward indexing, this will print "World"  
print(str2)  
str = str.replace("World", "Everyone")  
# due to the replace function, " Hello, Everyone " will be printed  
print(str)  
str = str.lower()  
# the lowercase version " hello, everyone " will be printed  
print(str)  
str = str.strip()  
# without whitespaces, "hello, everyone" will be printed  
print(str)  
split_str = str.split(",")  
# the string has been split around the comma and stripped, so the second part "everyone" will be printed  
print(split_str[1].strip())
```

```
↵ Hello, World  
Say Hello  
World  
Hello, Everyone  
hello, everyone  
hello, everyone  
everyone
```